

¿Firebase o RevenueCat para administrar las suscripciones?

Comparativa entre mantener el cobro/verificación en RevenueCat y hacerlo íntegramente sobre Firebase, ahora que el login de la app ya se está construyendo sobre esa plataforma. Incluye una tercera alternativa híbrida y una recomendación.

Preparado para: Ezequiel Castiglione

Proyecto: RallyStats (rally_stats)

Fecha: 25 de agosto de 2026

Estado: propuesta para discusión — sin cambios aplicados al código

Complementa el informe: "Suscripción mensual vía Google Play Store"

Índice

1. Resumen ejecutivo
2. La pregunta central: ¿Firebase tiene un producto de suscripciones equivalente a RevenueCat?
3. Qué significa realmente "hacerlo con Firebase"
4. Propuesta A — Firebase (login) + RevenueCat (suscripciones)
5. Propuesta B — Todo en Firebase (login + suscripciones a medida)
6. Propuesta C — Híbrido: RevenueCat + extensión oficial hacia Firestore
7. Comparativa detallada
8. Relación con la lista "VIP" (cuentas sin pago)
9. Reversibilidad: qué tan atada queda la decisión
10. Recomendación
11. Qué cambiaría la recomendación
12. Próximos pasos

1. Resumen ejecutivo

La pregunta original era si Firebase puede reemplazar a RevenueCat para la administración de las suscripciones, de forma de tener todo en una sola plataforma. La respuesta corta es: **Firestore no tiene un producto propio de suscripciones para Google Play** — no hay nada equivalente a RevenueCat dentro de Firebase. Lo que sí es viable es construir esa lógica a mano sobre la infraestructura de Firebase (Cloud Functions + Firestore), que de todas formas se va a sumar al proyecto por el login. Y hay una tercera opción, menos obvia, que probablemente resuelve mejor la intención real de la pregunta: usar RevenueCat para el cobro y su **extensión oficial de Firestore** para que todo el estado de la suscripción termine reflejado en Firestore y en el propio usuario de Firebase Auth, sin tener que elegir entre "un solo sistema" y "no programar la validación a mano".

RECOMENDACIÓN ADELANTADA

La propuesta C (RevenueCat + su extensión de sincronización a Firestore) es, para este proyecto, la que mejor equilibra "todo en una plataforma" con no tener que construir y mantener a mano la parte más riesgosa (la validación de pagos). Se desarrolla en la sección 10.

2. La pregunta central: ¿Firebase tiene un producto de suscripciones equivalente a RevenueCat?

Antes de comparar, conviene verificar el supuesto de la pregunta. Se revisó específicamente si existe una extensión oficial de Firebase/Google para validar suscripciones de Google Play (en el pasado circuló algo así bajo el nombre "Run Payments with Play Billing"). Buscando en el catálogo actual de Firebase Extensions Hub, **no aparece ninguna extensión oficial de Google para esto**. Lo que sí existe y está vigente:

Extensión	Qué hace	¿Sirve para reemplazar a RevenueCat?
RevenueCat — firebase-revenuecat-purchases	Recibe los webhooks de RevenueCat y sincroniza el estado de compra/entitlements de cada usuario a un documento de Firestore, y opcionalmente a un <i>custom claim</i> (<code>revenueCatEntitlements</code>) en el token de Firebase Auth	No — es un complemento de RevenueCat, no un sustituto. Da la integración con Firebase que se está buscando, sin sacar a RevenueCat del medio
Stripe — firebase-stripe-payments	Backend de pagos con Stripe sobre Firestore/Cloud Functions	No aplica: las políticas de Google Play exigen que las compras digitales dentro de una app Android pasen por Google Play Billing, no por Stripe
Purchasely — purchasely-in-app-purchases	Alternativa a RevenueCat (otro proveedor de terceros) con su propia extensión de Firebase	No — es la misma idea que RevenueCat, sólo cambia el proveedor

Conclusión: no hay atajo. "Hacerlo con Firebase" sin RevenueCat (ni ningún otro proveedor de terceros) significa programar la validación desde cero, tal como se describía como "backend propio" en el informe anterior — la diferencia es que ahora ese backend correría sobre Cloud Functions de Firebase en vez de un servidor aparte, porque de todas formas va a existir un proyecto de Firebase por el login.

Fuentes consultadas: [Firebase Extensions Hub — RevenueCat](#), [GitHub — RevenueCat/firebase-revenuecat-purchases](#), [RevenueCat — Firebase integration docs](#), [Firebase Extensions Hub — Stripe](#).

3. Qué significa realmente "hacerlo con Firebase"

Sin un producto nativo de suscripciones, "administrar todo con Firebase" se traduce en construir, con piezas genéricas de Firebase, lo mismo que ya describía el informe anterior como la ruta de backend propio:

- **Cloud Functions** con un endpoint HTTP que reciba las notificaciones de Google (Real-time Developer Notifications) publicadas en un tópico de Pub/Sub.
- Esa función llama a la **Google Play Developer API** (`purchases.subscriptions.get`) para confirmar el estado real de cada suscripción y lo escribe en **Firestore** (por ejemplo, una colección `subscriptions` con un documento por UID de Firebase).
- La app lee ese documento (o, mejor, un *custom claim* propio seteado con el Admin SDK) para decidir si desbloquea o bloquea funcionalidad.
- Hay que programar a mano el manejo de *todos* los tipos de evento de RTDN (renovación, cancelación, período de gracia, on hold, pausa, reactivación, reembolso, cambio de precio, etc.) y la idempotencia de esos webhooks.

Es exactamente el mismo trabajo de ingeniería descrito en la sección 6 del informe anterior — sólo que alojado en la infraestructura de Firebase en vez de un servidor propio aparte. No es menos trabajo por estar en Firebase; es el mismo trabajo, con menos piezas de infraestructura nuevas que aprender porque el proyecto de Firebase ya va a existir.

4. Propuesta A — Firebase (login) + RevenueCat (suscripciones)

Dos plataformas, cada una responsable de lo suyo:

Firebase Auth + Firestore

Login, control de dispositivo único
(`account_devices`), y en el futuro la lista VIP.

+

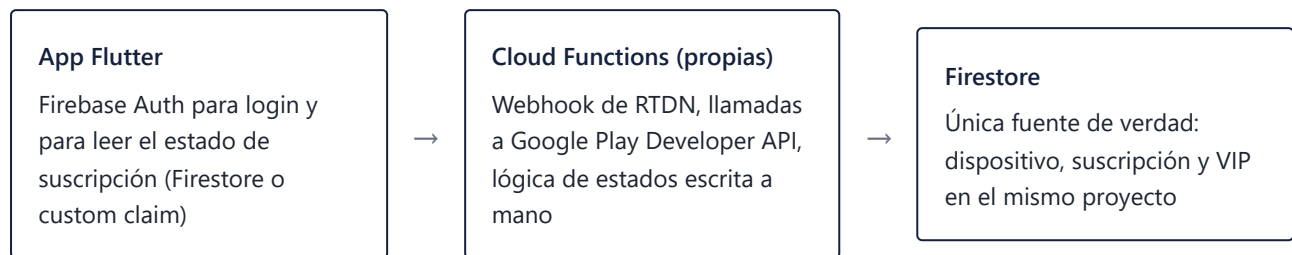
RevenueCat

Cobro, verificación de pagos, estados de
suscripción, entitlements, dashboard de métricas.

La única integración necesaria entre ambas es usar el **UID de Firebase como App User ID de RevenueCat** (`Purchases.login(firebaseUid)`) para que cada cuenta quede identificada igual en los dos sistemas. Es lo que ya se planteaba en el informe anterior y en la conversación sobre la lista VIP.

Ventaja: RevenueCat resuelve toda la parte de mayor riesgo (validación, manejo de estados, dashboard) sin código propio. **Desventaja frente al pedido actual:** son dos plataformas separadas — dos paneles, dos lugares donde mirar el estado de una cuenta.

5. Propuesta B — Todo en Firebase (login + suscripciones a medida)



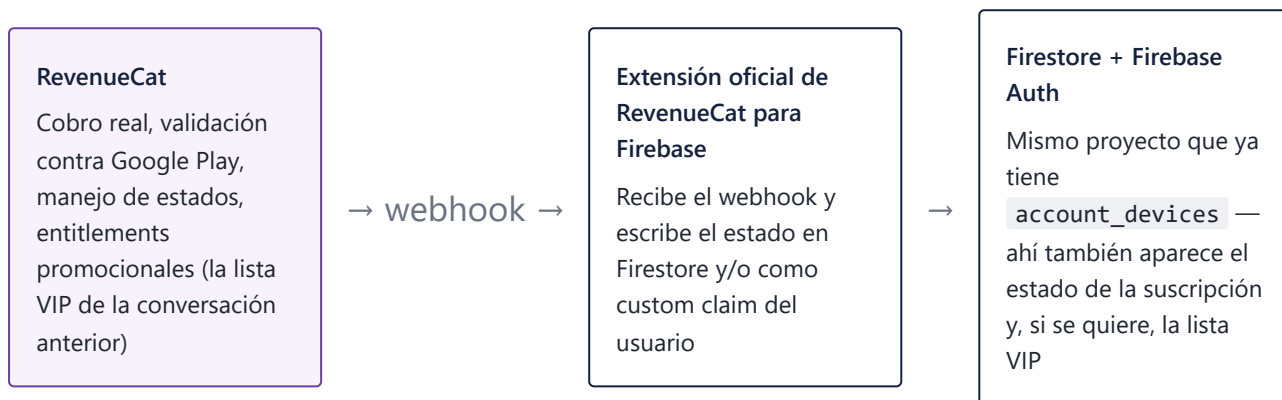
Es la propuesta que más literalmente cumple "todo integrado en una sola plataforma": un solo proyecto de Firebase, una sola consola, sin dependencia de terceros ni reparto de ingresos. El costo es que toda la lógica de validación de pagos —la parte más delicada, porque un error ahí bloquea a alguien que sí pagó o deja pasar a alguien que no pagó— queda enteramente a cargo del desarrollo propio, sin nada probado en producción por miles de apps detrás, como sí tiene RevenueCat.

PUNTO DE ATENCIÓN

Google agrega y modifica tipos de notificación de RTDN con el tiempo. Con backend propio, mantenerse al día con esos cambios es responsabilidad del proyecto; con RevenueCat, es RevenueCat quien lo mantiene actualizado para todos sus clientes.

6. Propuesta C — Híbrido: RevenueCat + extensión oficial hacia Firestore

Esta es la opción que no era obvia hasta revisar qué extensiones existen realmente (sección 2). Usa RevenueCat para todo el trabajo pesado de cobro y validación, pero agrega su extensión oficial de Firebase para que el resultado quede reflejado directamente en el mismo proyecto de Firebase que ya maneja el login:



Con el custom claim `revenueCatEntitlements` ya viajando en el token de Firebase Auth, la app puede resolver "¿está activa la suscripción?" leyendo el mismo token que ya usa para el login, sin una llamada de red aparte a RevenueCat en cada apertura. Y las reglas de seguridad de Firestore pueden usar ese mismo claim para proteger datos, exactamente con el mismo mecanismo que ya protege `account_devices`.

POR QUÉ IMPORTA PARA LA PREGUNTA ORIGINAL

Esto le da al proyecto un único lugar operativo (Firestore/Firebase) para mirar el estado de cualquier cuenta — login, dispositivo, suscripción y VIP — sin renunciar a que la validación de pagos la siga haciendo un sistema probado (RevenueCat) en vez de código propio sin testear en producción.

7. Comparativa detallada

	A — Firebase + RevenueCat	B — Todo en Firebase (a medida)	C — RevenueCat + extensión a Firebase
Esfuerzo de implementación	Bajo	Alto	Bajo/medio (SDK + instalar y configurar una extensión)
Mantenimiento continuo	Bajo — RevenueCat absorbe los cambios de Google	Alto — hay que seguir de cerca los cambios de RTDN/Play Developer API	Bajo — mismo motor que la opción A
Costo económico	Gratis hasta cierto volumen de ingresos, después % — revisar umbral vigente	Sólo hosting de Cloud Functions/Firestore (típicamente bajo a esta escala)	Igual que A + costo marginal casi nulo de la extensión (corre en Cloud Functions)
Todo en un solo panel/plataforma	No — dos consolas separadas	Sí — un solo proyecto de Firebase	Parcial/sí en la práctica — los datos terminan unificados en Firestore/Firebase Auth, aunque el cobro en sí se gestiona desde el panel de RevenueCat
Riesgo de bugs en la lógica de cobro	Bajo — motor probado por terceros	Alto — cada caso límite (gracia, on hold, reembolsos) se prueba por primera vez acá	Bajo — mismo motor que A
Dashboard / métricas de suscripción (MRR, churn, etc.)	Sí, integrado	No — habría que construirlo aparte	Sí, integrado (viene de RevenueCat)
Dependencia de un tercero	Sí	No	Sí (misma que A)

8. Relación con la lista "VIP" (cuentas sin pago)

Retomando la pregunta de la conversación anterior, así cambia el mecanismo VIP según la propuesta:

Propuesta	Cómo se otorgaría VIP
A — Firebase + RevenueCat	Entitlement promocional desde el dashboard de RevenueCat, atado al UID de Firebase usado como App User ID. Administración en un panel externo.
B — Todo en Firebase	Un documento más en una colección propia de Firestore (por ejemplo <code>vip_accounts</code>), gestionado enteramente a mano por el proyecto.
C — Híbrido	Se puede seguir usando el entitlement promocional de RevenueCat (mismo mecanismo que A) y, gracias a la extensión, ese otorgamiento también queda reflejado automáticamente en Firestore/custom claims — se administra desde el dashboard de RevenueCat pero se <i>consulta</i> desde el mismo lugar que todo lo demás (Firebase).

9. Reversibilidad: qué tan atada queda la decisión

Ninguna de las tres opciones es definitiva si el resto de la app (el `SubscriptionController` o equivalente) sólo le pregunta a una capa de verificación "¿está activo el entitlement de esta cuenta?" sin saber de dónde sale esa respuesta. Bajo ese diseño, migrar de A/C a B (o viceversa) más adelante es un cambio acotado a esa capa de verificación, no una reescritura de las pantallas de bloqueo/desbloqueo. Esto baja el costo de "elegir mal" ahora.

10. Recomendación

RECOMENDACIÓN

Propuesta C: Firebase para login (como ya está en marcha) + RevenueCat para el cobro/verificación + la extensión oficial de RevenueCat para Firebase, de forma que el estado de la suscripción también quede reflejado en Firestore y en el token de Firebase Auth.

Es la que mejor responde a la intención real de la pregunta ("todo integrado en una plataforma") sin asumir el costo de construir y mantener, desde cero y en un proyecto chico, la parte más sensible del sistema: la validación de que un pago efectivamente se procesó.

La propuesta B (todo hecho a mano en Firebase) es técnicamente viable y queda como alternativa válida si en algún momento pesa más evitar por completo la dependencia de un tercero (o su costo a medida que crezcan los ingresos) que el tiempo de desarrollo y el riesgo de mantener esa lógica en el tiempo. Pero no es "más simple por estar en Firebase": es el mismo nivel de esfuerzo que ya se había descartado como la opción de mayor costo en el informe anterior.

11. Qué cambiaría la recomendación

Si el volumen de ingresos crece lo suficiente como para que el % de RevenueCat sea significativo

Ahí migrar la validación a Cloud Functions propias (propuesta B) empieza a justificarse económicamente — y es un cambio acotado gracias a la capa de abstracción mencionada en la sección 9.

Si en algún momento se suma otra plataforma de pago (iOS, Stripe para la versión Windows, etc.)

Ahí el valor de RevenueCat (abstraer múltiples tiendas bajo una misma API) aumenta — refuerza la propuesta C/A en vez de debilitarla.

Si la prioridad es cero dependencias externas por una razón de política/control, no de costo

Ahí la propuesta B es la que corresponde, aceptando el mayor esfuerzo de desarrollo y mantenimiento como costo consciente.

12. Próximos pasos

- Confirmar si la prioridad detrás de "todo en una plataforma" es operativa (un solo lugar donde mirar) o económica/de control (evitar terceros) — determina si C o B es la respuesta correcta.
- Si se confirma la propuesta C: sumar al login con Firebase (ya en curso) el paso de `Purchases.login(firebaseUid)` al integrar RevenueCat, e instalar su extensión de Firebase cuando se implemente la suscripción.
- Revisar en conjunto el informe anterior ("Suscripción mensual vía Google Play Store"), en particular las preguntas abiertas de su sección 14, que siguen vigentes sea cual sea la opción elegida acá.